

Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.

```
import numpy as np
import matplotlib.pyplot as plt
```

```
hours = np.array([1,2,3,4,5,6,7,8,9,10])
scores = np.array([60,70,80,90,100,110,120,130,140,150])
```

```
predicted_score=10*5+50
print("Best Fit line Equation:")
print("score = 10*hours +50")
print("\nFor 5 hours -> predicted score:",predicted_score)
```

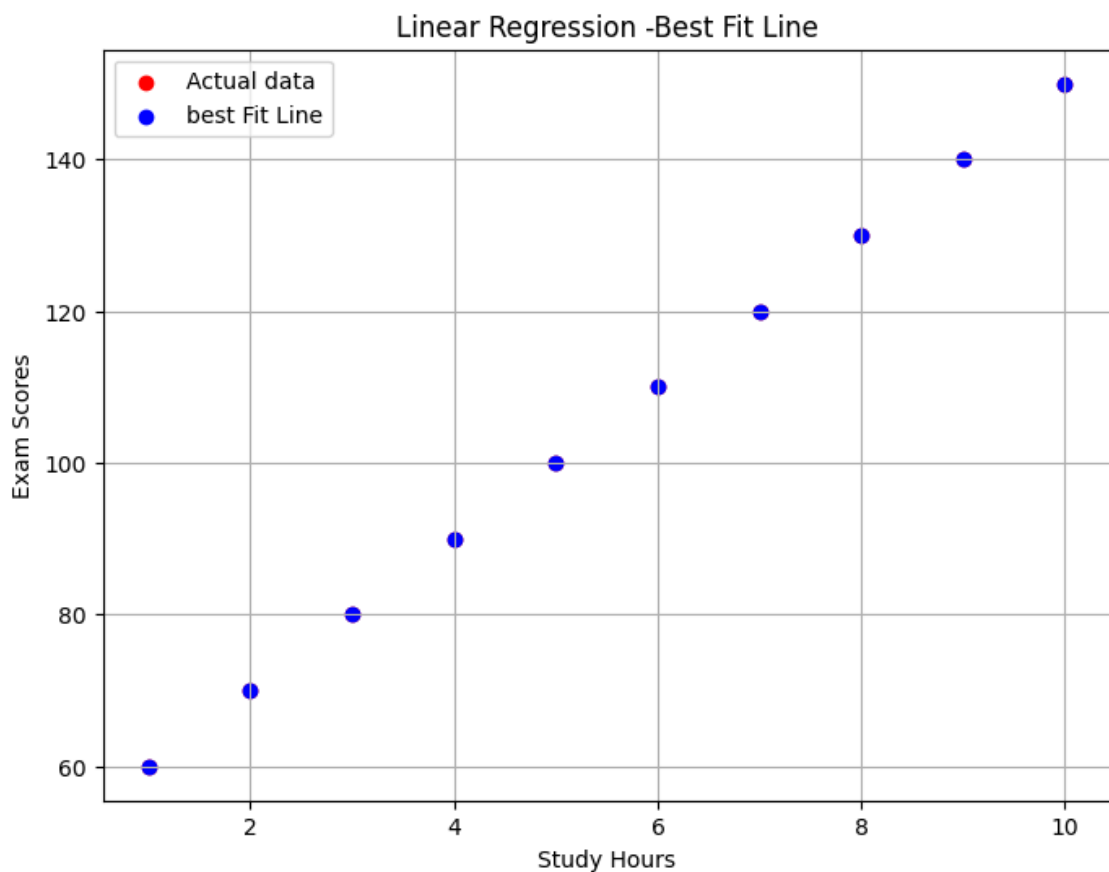
Best Fit line Equation:
score = 10*hours +50

For 5 hours -> predicted score: 100

```
import numpy as np
import matplotlib.pyplot as plt
predicted_score=10*5+50

hours = np.array([1,2,3,4,5,6,7,8,9,10])
scores = np.array([60,70,80,90,100,110,120,130,140,150])

predicted_scores=10* hours+50
plt.figure(figsize=(8,6))
plt.scatter(hours,scores,label="Actual data",color="red")
plt.scatter(hours,predicted_score,label="best Fit Line",color="blue")
plt.scatter(hours,predicted_scores,label="best Fit Line",color="blue")
plt.xlabel("Study Hours")
plt.ylabel("Exam Scores")
plt.title("Linear Regression -Best Fit Line")
plt.legend()
plt.grid(True)
plt.show()
```



◆ Gemini

```
import numpy as np
x= np.array([1,2,3,4,5], dtype=float)
y= np.array([60,70,80,90,100],dtype=float)
m=0
c=0
learning_rate=0.01
epochs=1000
n=len(x)
for i in range(epochs):
    y_pred=m*x+c
    error=y-y_pred
    mse = np.mean(error**2)
    dm=(-2/n)*np.sum(x*error)
    dc=(-2/n)*np.sum(error)
    m=m-(learning_rate*dm)
    c=c-(learning_rate*dc)
    if i%100==0:
        print(f"epoch{i}")
        print(f"MSE = {mse:.2f}")
        print(f"m={m:.2f}, c={c:.2f}")
        print("-----")
print("\nFinal Best Fit Line:")
print(f"y={m:.2f}x+{c:.2f}")
```

```
epoch0
MSE =6600.00
m=5.20,c=1.60
-----
epoch100
MSE =177.90
m=18.63,c=18.84
-----
epoch200
MSE =90.37
m=16.15,c=27.79
```

```

-----
epoch300
MSE =45.90
m=14.38,c=34.17
-----
epoch400
MSE =23.32
m=13.12,c=38.72
-----
epoch500
MSE =11.84
m=12.23,c=41.96
-----
epoch600
MSE =6.02
m=11.59,c=44.27
-----
epoch700
MSE =3.06
m=11.13,c=45.92
-----
epoch800
MSE =1.55
m=10.81,c=47.09
-----
epoch900
MSE =0.79
m=10.57,c=47.93
-----
/nFinal Best Fit Line:
y=10.41x+48.52

```

```

from sklearn.model_selection import train_test_split
import numpy as np

experience = np.array([1,2,3,4,5,6,7,8,9,10,11,12,13,14,15])
salary = np.array([35,38,42,48,55,60,69,70,76,82,87,92,96,100,105])

x = experience.reshape(-1, 1)
y = salary

x_train, x_test, y_train, y_test = train_test_split(
    x,y,
    test_size=0.2,
    random_state=42
)

print(f"Training samples : {len(x_train)}")
print(f"Training samples : {len(x_test)}")

```

```

Training samples : 12
Training samples : 3

```

```
from sklearn.linear_model import LinearRegression
```

```

model = LinearRegression()
model.fit(x_train,y_train)

```

▼ LinearRegression ⓘ ?

```
LinearRegression()
```

```

print(f"slope (cofficient) : {model.coef_[0]:.2f}")
print(f"intercept : {model.intercept_:.2f}")

```

```

slope (cofficient) : 5.20
intercept : 28.44

```

```
y_pred = model.predict(x_test)
```

```
for actual, predicted in zip(y_test,y_pred):
    print(f"Actual: {actual:>5} | Predicted : {predicted:>7.1f}| "f"Error:{predicted - actual:+.1f}")
```

Actual:	82	Predicted :	80.5	Error:-1.5
Actual:	92	Predicted :	90.9	Error:-1.1
Actual:	35	Predicted :	33.6	Error:-1.4

```
from sklearn.metrics import r2_score,mean_absolute_error,mean_squared_error
import numpy as np
```

```
y_actual = np.array([500,300,700,450,600])
y_pred = np.array([480,320,690,460,590])
```

```
r2 = r2_score(y_actual,y_pred)
mae = mean_absolute_error(y_actual,y_pred)
rmse = np.sqrt(mean_squared_error(y_actual,y_pred))
```

```
print(f"R2 Score : {r2:.4f} → model explains {r2*100:.1f}% of variance")
print(f"MAE : {mae:.1f} → avg error ± {mae:.0f} units")
print(f"RMSE : {rmse:.1f} → penalised avg error {rmse:.0f} units")
```

```
R2 Score : 0.9880 → model explains 98.8% of variance
MAE : 14.0 → avg error ± 14 units
RMSE : 14.8 → penalised avg error 15 units
```

```
import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score,mean_absolute_error
import numpy as np
```

```
np.random.seed(42)
n = 10
data = pd.DataFrame({
    'experience':np.random.randint(0,20,n),
    'education':np.random.randint(12,20,n),
    'age':np.random.randint(22,55,n),
})
```

```
data['salary'] =(
    data['experience']*4500+
    data['education']*2000+
    data['age']*300+
    np.random.normal(0,3000,n)+
    10000
)
```

```
x = data[['experience','education','age']]
y = data['salary']
x_train, x_test, y_train, y_test = train_test_split(
    x,y,test_size=0.2,random_state=42)
model = LinearRegression()
model.fit(x_train,y_train)
```

▼ LinearRegression ⓘ ?

LinearRegression()

```
for feat, coef in zip(x.columns,model.coef_):
    print(f"{feat:12s} : ${coef:,.0f} per unit increase")
    print(f" Base salary (intercept): ${model.intercept_:,.0f}")
    y_pred = model.predict(x_test)
    print(f"\nR2 : {r2_score(y_test,y_pred):.3f}")
    print(f"MAE : {mean_absolute_error(y_test,y_pred):,.0f}")
```

```
experience   : $4,402 per unit increase
Base salary (intercept): $19,954
```

```
R2 : 0.995
```

```
MAE : 1,587
```

```
education    : $1,650 per unit increase
Base salary (intercept): $19,954
```

```
R2 : 0.995
```

```
MAE : 1,587
```

```
age          : $207 per unit increase
Base salary (intercept): $19,954
```

```
R2 : 0.995
```

```
MAE : 1,587
```

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
```

```
File "/tmp/ipykernel_418/4227844237.py", line 3
    from sklearn.linear_model import Linear Regression
                                         ^
```

SyntaxError: invalid syntax

Next steps: [Explain error](#)

```
import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
np.random.seed(42)
n = 200
locations = ['Urban', 'Suburban', 'Rural']
df = pd.DataFrame({
    'area_sqft': np.random.randint(700, 6000, n),
    'num_rooms': np.random.randint(1, 7, n),
    'location': np.random.choice(locations, n),
    'age_years': np.random.randint(0, 40, n),
})
loc_map = {'Urban': 1.5, 'Suburban': 1.0, 'Rural': 0.7}#expensiv medium cheap
noise = np.random.normal(0, 20000, n)
df['price'] = (
    df['area_sqft'] * 160 +
    df['num_rooms'] * 24000 -
    df['age_years'] * 3000 +
    df['location'].map(loc_map) * 40000 +
    noise
```

```

).astype(int)
print(df.head(8).to_string(index=False))
print(f"\nDataset shape : {df.shape}")
print(df.describe()[['area_sqft', 'num_rooms', 'age_years', 'price']].round(0))

```

area_sqft	num_rooms	location	age_years	price
1560	6	Urban	12	418474
5926	3	Suburban	39	964633
5891	1	Rural	8	972080
4472	1	Rural	26	696579
3792	4	Rural	1	740042
1166	3	Urban	4	295985
5126	6	Suburban	28	894554
4144	5	Suburban	36	689007

Dataset shape : (200, 5)

	area_sqft	num_rooms	age_years	price
count	200.0	200.0	200.0	200.0
mean	3419.0	3.0	19.0	617357.0
std	1526.0	2.0	12.0	243068.0
min	734.0	1.0	0.0	134739.0
25%	2192.0	2.0	9.0	426976.0
50%	3432.0	3.0	18.0	615756.0
75%	4603.0	5.0	29.0	835531.0
max	5979.0	6.0	39.0	1156048.0

```

le = LabelEncoder()
#transform Converts them into numbers automatically
df['location_enc'] = le.fit_transform(df['location'])
# Select features and target
features = ['area_sqft', 'num_rooms', 'age_years', 'location_enc']
X = df[features]
y = df['price']
# Peek at encoded data
print(df[['location', 'location_enc']].drop_duplicates().sort_values('location_enc'))
print(f"\nFeature matrix shape : {X.shape}")
print(f"Target vector shape : {y.shape}")

```

	location	location_enc
2	Rural	0
1	Suburban	1
0	Urban	2

Feature matrix shape : (200, 4)

Target vector shape : (200,)

```

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
print(f"Training samples : {len(X_train)} ({len(X_train)/len(X)*100:.0f}%)")
print(f"Testing samples : {len(X_test)} ({len(X_test)/len(X)*100:.0f}%)")
print(f"Feature names : {features}")

```

Training samples : 160 (80%)

Testing samples : 40 (20%)

Feature names : ['area_sqft', 'num_rooms', 'age_years', 'location_enc']

```

model = LinearRegression()
model.fit(X_train, y_train)
print("Learned Coefficients:")
print("-" * 45)
for feat, coef in zip(features, model.coef_):
    direction = "↑" if coef > 0 else "↓"
    print(f" {feat:15s}: Rs {coef:>10,.0f} {direction}")
print(f" {'intercept':15s}: Rs {model.intercept_:>10,.0f}")
print("-" * 45)

```

Learned Coefficients:

```

area_sqft      : Rs      151 ↑
num_rooms      : Rs     23,169 ↑
age_years      : Rs    -2,262 ↓
location_enc    : Rs     18,114 ↑
intercept      : Rs     46,789

```

```

y_pred = model.predict(X_test)
r2 = r2_score(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
print("=" * 40)
print(" MODEL EVALUATION REPORT")
print("=" * 40)
print(f"R² Score : {r2:.4f} {'✓ Excellent' if r2>0.9 else '✓ Good' if r2>0.8 else '■ Needs work'}")
print(f"MAE : Rs {mae:>10,.0f}")
print(f"RMSE : Rs {rmse:>10,.0f}")
print("=" * 40)
# Show first 8 predictions vs actuals
print("\nSample Predictions:")
print(f"{'Actual':>10} | {'Predicted':>10} | {'Error':>10}")
print("-" * 36)
for a, p in zip(y_test[:8], y_pred[:8]):
    print(f"{a:>10,} | {p:>10,.0f} | {p-a:>10,.0f}")

```

```

=====
MODEL EVALUATION REPORT
=====
R² Score : 0.9872 ✓ Excellent
MAE : Rs      13,439
RMSE : Rs     16,959
=====

```

```

Sample Predictions:

```

Actual	Predicted	Error
489,899	478,320	-11,579
601,329	572,877	-28,452
623,788	650,489	+26,701
291,458	271,086	-20,372
390,040	344,431	-45,609
123,884	134,545	+10,661
484,442	443,295	-41,147
562,823	556,350	-6,473

```

new_houses = pd.DataFrame({
    'area_sqft': [1100, 3000, 600, 5000],
    'num_rooms': [5, 8, 1, 9 ],
    'age_years': [8, 11, 13, 2 ],
    'location_enc':[3, 6, 1, 2 ], # 2=Urban,1=Suburban,0=Rural
    'location_lbl':['Urban', 'Suburban', 'Rural', 'Urban'],
})
predictions = model.predict(new_houses[features])
print("\n HOUSE PRICE PREDICTIONS")
print("=" * 60)
for i, (_, row) in enumerate(new_houses.iterrows()):
    print(f" House {i+1}: {row.area_sqft} sqft | "
          f"{row.num_rooms} rooms | "
          f"{row.age_years}yr old | "
          f"{row.location_lbl}")
    print(f" Estimated Price → Rs {predictions[i]:>10,.0f}")
    print()

```

HOUSE PRICE PREDICTIONS

Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.